

Nome: Eduardo Lucas Lemes Januário CT3037568

Nome: Guilherme Batista CT3037274

Trabalho Teórico - Estrutura de Dados 2

Exercício 1 – Soma de Vetor (Iterativo)

Considere o seguinte algoritmo que calcula a soma dos elementos de um vetor de tamanho n :

```
def soma_vetor(A):  
    total = 0  
    for i in range(len(A)):  
        total += A[i]  
    return total
```

- Determine a complexidade de tempo em função de n .
- Justifique sua resposta.

$$T(n) = T(n - 1) + c$$

$$T(n) = T(n - 2 + c) + c$$

$$T(n) = T(n - 2) + 2c$$

$$T(n) = T(n - 3 + c) + 2c$$

$$T(n) = T(n - 3) + 3c$$

Resultando em:

$$T(n) = T(n - k) + kc$$

Com base $T(1)$

$$n - k = 1 \rightarrow k = n - 1$$

$$T(n) = T(n - (n - 1)) + (n - 1)c$$

$$T(n) = T(n - n + 1) + cn - c$$

$$T(n) = T(1) + cn - c$$

Retirando as constantes, temos $\rightarrow O(n)$

Exercício 2 – Complexidade do Merge Sort (Recursivo)

- Escreva a recorrência do Merge Sort
- Resolva a recorrência e determine a complexidade de tempo do algoritmo.

$$T(n) = T(n/2) + T(n/2) + c * n$$

$$T(n) = 2T(n/2) + c * n$$

$$T(n/2) = 2T(n/4) + c * (n/2)$$

$$T(n) = 2(2T(n/4) + c * (n/2)) + c * n$$

$$T(n) = 4T(n/4) + 2c * (n/2) + c * n$$

$$T(n) = 4T(n/4) + 2c * n$$

$$T(n) = 2^k * T(n/2^k) + k(cn)$$

$$n/2^k = 1 \rightarrow 2^k = n \rightarrow k = \log_2 n$$

$$T(n) = n * T(1) + (\log_2 n) * c * n$$

Portanto $O(n \log n)$

- Explique passo a passo como o Merge Sort divide e combina os subvetores.

O Merge Sort é um algoritmo de ordenação por divisão e conquista. Primeiramente ele divide o vetor em duas metades, em seguida ele ordena recursivamente cada metade, e logo após ele combina as metades ordenadas em um único vetor ordenado. Vale ressaltar que a divisão ocorre até sobrar um elemento, antes de eles serem comparados, ordenados e unidos em subvetores.

Exercício 3 – Complexidade do Selection Sort (Iterativo)

Considere o algoritmo Selection Sort:

```
def selection_sort(A):
    n = len(A)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if A[j] < A[min_idx]:
                min_idx = j
        A[i], A[min_idx] = A[min_idx], A[i]
```

- Determine a complexidade de tempo (melhor, médio e pior caso).

Pior Caso: $O(n^2)$

Melhor Caso: $O(n)$

Caso Médio: $O(n^2)$

- Explique por que a complexidade não muda dependendo do estado inicial do vetor.

A complexidade do Selection Sort não muda porque o número de comparações é mesmo o mesmo. O estado inicial do vetor só afeta o número de trocas, que não altera a ordem da complexidade.

Exercício 4 – Busca Linear vs Busca Binária

Considere dois algoritmos de busca em um vetor ordenado de tamanho n :

- Busca Linear: percorre todos os elementos até encontrar o alvo.
- Busca Binária: divide o vetor ao meio a cada passo (recursiva ou iterativa).
- Escreva a complexidade de tempo para ambos os algoritmos (melhor, médio e pior caso).

	Busca Linear	Busca Binária
Melhor Caso	$O(1)$	$O(1)$
Caso Médio	$O(n)$	$O(\log n)$
Pior Caso	$O(n)$	$O(\log n)$

- Explique por que a busca binária é muito mais eficiente para vetores grandes. A busca binária é mais eficiente para vetores grandes, porque reduz o número de comparações em grande escala.

Exercício 5 – Bubble Sort (Iterativo)

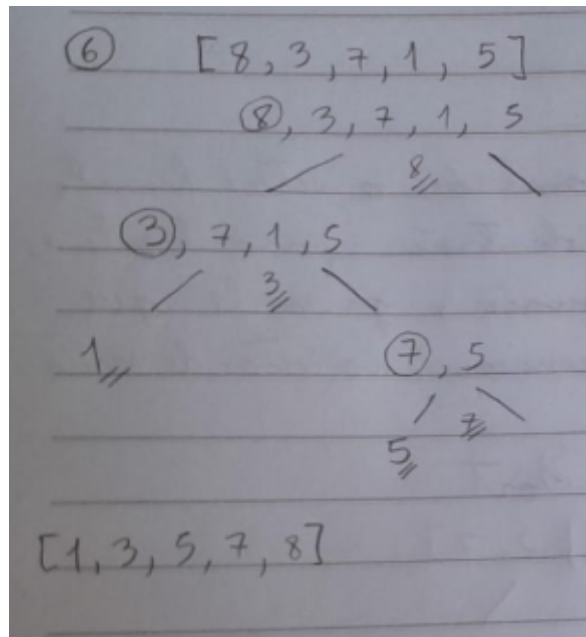
- Demonstre o funcionamento do Bubble Sort para o vetor $[5, 2, 9, 1, 6, -1, 7]$. Mostre passo a passo como os elementos são comparados e trocados.

⑤ $[5, 2, 9, 1, 6, -1, 7]$ Bubble Sort

$\underline{5}, 2, 9, 1, 6, -1, 7$
 $2, \underline{5}, 9, 1, 6, -1, 7$
 $2, 5, \underline{9}, 1, 6, -1, 7$
 $2, 5, 1, \underline{9}, 6, -1, 7$
 $2, 5, 1, 6, \underline{9}, -1, 7$
 $2, 5, 1, 6, -1, \underline{9}, 7$
 $2, 5, 1, 6, -1, 7, \underline{9}$
 $2, \underline{5}, 1, 6, -1, 7, 9$
 $2, 1, 5, \underline{6}, -1, 7, 9$
 $3, \underline{1}, 5, -1, 6, 7, 9$
 $1, 2, \underline{5}, -1, 6, 7, 9$
 $1, 2, -1, \underline{5}, 6, 7, 9$
 $\underline{1}, -1, 2, 5, 6, 7, 9$
 $-1, 1, 2, 5, 6, 7, 9$
 $[-1, 1, 2, 5, 6, 7, 9]$

Exercício 6 – Quick Sort (Recursivo)

- Demonstre o funcionamento do Quick Sort para o vetor [8, 3, 7, 1, 5]. Mostre como o pivô é escolhido, como o vetor é dividido em subvetores e como eles são combinados.



- Escreva um pseudocódigo resumido do algoritmo.

Função Quick Sort(lista)

Se Tamanho(lista) ≤ 1 então

retornar lista

FimSe

pivo $\leftarrow$ lista[0]

menores $\leftarrow$ lista vazia

maiores $\leftarrow$ lista vazia

Para cada elemento x em lista[a até fim] faça

Se x $\leq$ pivo então

adicionar x a menores

Senão

adicionar x a maiores

FimSe

FimPara

retornar Concatenar(QuickSort(menores), [pivo], QuickSort(maiores))

FimFunção

- Explique a complexidade de tempo nos casos melhor e pior.
Melhor Caso: $O(n \log n)$, quando o pivô divide bem o vetor
Pior Caso: $O(n^2)$, quando o pivô é sempre o maior ou menor elemento

Exercício 7 – Counting Sort (Estável)

- Explique como o Counting Sort funciona, incluindo o vetor auxiliar de contagem cumulativa usado para manter a estabilidade.

O Counting Sort é um algoritmo de ordenação que utiliza um vetor auxiliar para contar a frequência de cada elemento do vetor de entrada. Ao final, os valores são inseridos na ordem correta usando a contagem e o índice da contagem do vetor auxiliar.

- Demonstre o algoritmo para o vetor [4, 2, 2, 8, 3, 3, 1] passo a passo, mostrando como o vetor de contagem é construído e como os elementos são posicionados no vetor final.

[4, 2, 2, 8, 3, 3, 1] → [0 a 8]

0 1 2 3 4 5 6 7 8

Vetor Auxiliar → [0, 1, 2, 2, 1, 0, 0, 0, 1]

Vetor Final → [1, 2, 2, 3, 3, 4, 8]

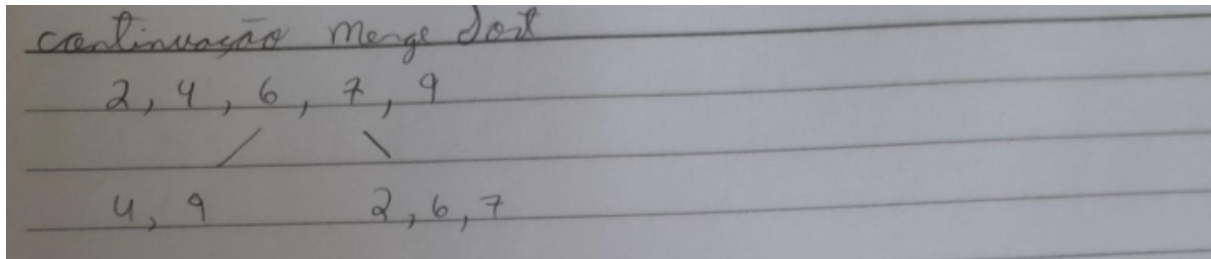
- Justifique por que o algoritmo é estável.

O algoritmo é estável porque ele constrói o vetor final, percorrendo o vetor de entrada de trás para frente, e quando encontra elementos iguais, garante que não sejam colocados em posições sucessivas no vetor de saída.

Exercício 8 – Ordenação por Seleção e Merge

- Para o vetor [9, 4, 6, 2, 7], mostre passo a passo como o Selection Sort ordena o vetor.
- Para o mesmo vetor, demonstre o Merge Sort passo a passo.

The image shows handwritten notes on lined paper illustrating two sorting algorithms. On the left, under the heading '⑧ Selection Sort', the steps for sorting the array [9, 4, 6, 2, 7] are shown. The minimum element is identified and swapped with the first element in each iteration, resulting in the sorted array [2, 4, 6, 7, 9]. On the right, under the heading 'Merge Sort', the recursive splitting of the array [9, 4, 6, 2, 7] into sub-arrays [9, 4] and [6, 2, 7] is shown, followed by the merging process back into the sorted array [2, 4, 6, 7, 9]. A 'tilibra' logo is visible in the bottom left corner of the paper.



- Compare as duas abordagens e explique suas diferenças em termos de complexidade e funcionamento.

O algoritmo Selection Sort é um algoritmo simples, com complexidade $O(n^2)$, o qual encontra o menor elemento e o coloca na posição correta, mas faz comparações desnecessárias, sendo ineficiente para grandes conjuntos de dados.

O algoritmo Merge Sort é mais complexo, com complexidade $O(n \log n)$, já que divide o vetor em partes menores, ordena cada uma delas e faz a junção novamente, sendo mais eficiente para grandes conjuntos de dados.